

Lista de Exercícios — Nível Intermediário

Programação em Python: Tratamento Avançado de Erros e Controle de Fluxo

CONTEÚDOS ABORDADOS

- **Exercício 1:** Validação Contínua de Entrada de Dados (`while`, `try-except` e `raise`).
- **Exercício 2:** Estrutura completa de controle (`try`, `except FileNotFoundError`, `else` e `finally`).
- **Exercício 3:** Processamento de Dados Misto e Captura Agrupada (`ValueError`, `TypeError`, `ZeroDivisionError`) as e.

EXERCÍCIO 1

Validação Contínua de Entrada de Dados

Crie uma função chamada `solicitar_idade()` que peça ao usuário para digitar sua idade no console. O programa deve garantir que a entrada seja totalmente válida antes de continuar a execução. Atenda aos seguintes requisitos:

1. **Repetição:** Utilize um loop `while` que continue solicitando a idade até que uma entrada correta seja fornecida.
2. **Conversão:** Tente converter o valor digitado para um número inteiro. Se o usuário digitar um texto ou caractere inválido (ex: "vinte"), capture o `ValueError` e exiba a mensagem: "Entrada inválida! Digite apenas números inteiros."
3. **Validação com raise:** Se a conversão for bem-sucedida, verifique se o número está entre 0 e 120. Caso esteja fora dessa faixa, lance manualmente uma exceção utilizando:

```
raise ValueError("Idade fora do intervalo permitido (0 a 120 anos).")
```

4. **Captura da Regra:** Capture também a mensagem do `raise` para orientar o usuário.
5. **Retorno:** Retorne o valor numérico da idade assim que for validado com sucesso.

EXERCÍCIO 2

Estrutura Completa de Controle

Escreva uma função chamada `processar_notas_arquivo(nome_arquivo)` que receba o nome de um arquivo de texto contendo notas de alunos (um número por linha) para calcular a média da turma. Aplique a estrutura completa de tratamento de exceções do Python:

1. **Bloco try:** Abra e leia todas as linhas do arquivo especificado em `nome_arquivo`, convertendo cada linha para `float` e armazenando em uma lista.
2. **Bloco except FileNotFoundError:** Se o arquivo especificado não existir no diretório, capture a exceção e exiba uma mensagem informativa ao usuário: "Erro: O arquivo informado não foi encontrado no sistema."
3. **Bloco else:** Caso o arquivo seja lido sem nenhum erro, calcule a média aritmética das notas contidas na lista e imprima o resultado formatado (ex: "Média da turma: 8.5").
4. **Bloco finally:** Exiba sempre a mensagem "Operação de leitura finalizada.", garantindo que ela seja executada independentemente de ter ocorrido erro ou sucesso no processamento.

EXERCÍCIO 3

Processamento de Dados Misto e Captura Agrupada

Uma API legada retornou uma lista com dados desordenados e de tipos variados contendo números, strings, listas e valores nulos:

```
dados_brutos = [10, "20", "trinta", 0, [5, 5], 50, None, "100"]
```

Escreva uma função chamada `calcular_inversos(lista_dados)` que percorra cada elemento dessa lista e tente calcular a divisão `100 / int(item)`:

1. **Captura Agrupada:** Utilize um **único bloco except** agrupando em uma tupla as três exceções possíveis:

```
except (ValueError, TypeError, ZeroDivisionError) as e:
```

2. **Detalhamento do Erro:** Para cada item que falhar na conversão ou no cálculo, imprima o valor original do item e a mensagem nativa capturada pelo alias `e` (ex: `f"Falha ao processar o item '{item}': {e}"`).
3. **Armazenamento de Sucessos:** Para os itens processados com sucesso, calcule o resultado da divisão e armazene-o em uma nova lista.
4. **Retorno:** Retorne a lista contendo apenas os resultados válidos ao final do processamento.

